

5/11/2025 0120 hrs

pseudocode:

```
activateNode(){
    it = 0;
    while( it < MAX_ITR ) do {
        /* wait till exchange operation is initiated by the parent thread
*/
        lock(&xchg_init_lock);
        while( xchg_init == false ) do {
            cond_wait(&xchg_init_cv, &xchg_init_lock);
        }
        unlock(&xchg_init_lock);

        /* carry out receiving operation for adjacent nodes distance
vectors */
        receive();
        lock(&rcv_cnt_lock);
        rcv_cnt++;
        if( rcv_cnt == N ) { // total number of nodes
            cond_signal(&xchg_done);
        }
        unlock(&rcv_cnt_lock);

        /* wait till the updation operation is initiated by the parent
thread */
        lock(&updation_init_lock);
        while( updation_init == false ) do {
            cond_wait(&updation_init_cv, &updation_init_lock);
        }
        unlock(&updation_init_lock);

        /* carry out updation operation */
        update();
        lock(&update_cnt_lock);
        update_cnt++;
        if( update_cnt == N ) {
            cond_signal(&updation_done);
        }
    }
}
```

```

        unlock(&update_cnt_lock);

        it++;
    }

```

parent thread:

```

it = 0;
xchg_init = false;
updation_init = false;

create_and_start_all_node_threads();

while( it < MAX_ITR ) {

    // initiate exchange operation
    lock( &xchg_init_lock );
    xchg_init = true;
    cond_broadcast( &xchg_init_cv );
    unlock( &xchg_init_lock );

    // wait till all exchanges are completed
    lock( &xchg_cnt_lock );
    while( xchg_cnt < N ) do {
        cond_wait( &xchg_done, &xchg_cnt_lock );
    }
    xchg_cnt = 0;
    unlock( &xchg_cnt_lock );

    // initiate updation operation
    lock( &updation_init_lock );
    updation_init = true;
    cond_broadcast( &updation_init_cv );
    unlock( &updation_init_lock );

    // wait till all updations are completed
    lock( &updation_cnt_lock );
    while( updation_cnt < N ) do {
        cond_wait( &updation_done, &updation_cnt_lock );
    }
    updation_cnt = 0;
    unlock( &updation_cnt_lock );
}

```

```

// reset init booleans
lock( &xchg_init_lock );
    xchg_init = false;
unlock( &xchg_init_lock );

lock( &updation_init_lock );
    updation_init = false;
unlock( &updation_init_lock );

it++;
}

```

- `receive()` : stores the new distance vectors of the adjacent nodes (information exchange) in the calling node.
- `update()` : updates the distance vector of current node using the exchanged information.

Main problems in above pseudocode

1. Wrong receive completion check.

In `activateNode()` you do `if (rcv_cnt == N) { cond_signal(&xchg_done); }` – but a router should wait for messages from *its adjacent routers*, not from all `N` nodes. Using `N` will deadlock or produce incorrect synchronization for most topologies.

2. Inconsistent variable names / typos (will break compilation / reasoning):

- Parent unlock uses `unlock( &xcgh_init_lock );` (typo).
- Parent waits on `xchg_cnt` / signals `xchg_done` but worker increments `rcv_cnt` and signals `xchg_done`. The counter names and waiters/signallers must match exactly.

3. Wrong signalling primitive for many waiters.

You use `cond_signal()` when multiple threads may be waiting on the same condition variable. Use `cond_broadcast()` (or `signalAll` as you wrote in parent – be consistent). Single `cond_signal()` may wake only one waiter leaving others blocked.

4. Counters are never reset.

After a phase completes you must reset the shared counters (`rcv_cnt`, `update_cnt`, or phase counters) to `0` before the next iteration, while holding the associated lock; otherwise the next iteration's waits will not behave correctly.

5. Race between parent and workers on init flags and counters.

The parent sets `xchg_init = true` and broadcasts; workers increment counters and

possibly signal completion. The parent then waits on the completion variable – but if the parent checks before all increments/ signals happen or if counters are read without correct locking, subtle races occur. Use the counter lock consistently and do the wait with a `while (counter < needed)` loop.

6. Spurious wakeups handled OK in places but be consistent.

You use `while ( xchg_init == false ) cond_wait(...)` – good. Use the same pattern for other `cond_waits`.

7. You re-implement a barrier.

The pattern you want is a reusable barrier per iteration (two-phase: exchange and update). Implementing a correct barrier (mutex + counter + `cond_broadcast`) is simpler and less bug-prone than many ad-hoc variables.

8. Shared queue access unspecified.

Make sure `receive()` pops from the shared queue under a lock or using a thread-safe queue (or use per-edge queues). The spec requires routers to wait until they have received from *all adjacent routers* before updating.

Suggested corrected approach (concise, fixes above issues)

Use:

- per-node `adj_count` = number of neighbours for that node (from input).
- two reusable barriers (exchange_barrier, update_barrier) implemented with (mutex, counter, `cond_var`).
- use `cond_broadcast()` (or `signalAll`) to wake all waiting threads.
- reset counters inside the barrier.

Corrected pseudocode (C-style):

```
/* shared data */
bool stop = false;
int MAX_ITR;
shared_queue_t shared_queue; // thread-safe or guarded by its own mutex

/* reusable barrier type */
struct barrier {
    mutex_t lock;
    cond_t cv;
    int count;           // threads that have reached barrier
    int threshold;       // how many threads to wait for (varies per-phase)
```

```

};

void barrier_init(barrier *b, int threshold) {
    b->count = 0; b->threshold = threshold;
    init_mutex(&b->lock); init_cond(&b->cv);
}

void barrier_wait(barrier *b) {
    lock(&b->lock);
    b->count++;
    if (b->count == b->threshold) {
        b->count = 0;           // reset for next use
        cond_broadcast(&b->cv);
        unlock(&b->lock);
        return;
    }
    while (b->count != 0) {    // wait until leader resets count and broad
        cond_wait(&b->cv, &b->lock);
    }
    unlock(&b->lock);
}

/* per-node data */
struct node {
    char name;
    int adj_count;           // number of neighbours (from input)
    routing_table_t table;
    // other per-node locks if needed
};

/* global barriers: we'll reuse them each iteration */
barrier exchange_barrier; // threshold = total_nodes (or only nodes parti
barrier update_barrier;   // threshold = total_nodes

/* worker thread */
void activateNode(node *self) {
    int it = 0;
    while (it < MAX_ITR) {
        // Phase 1: wait until parent (or coordinator) says "start exchar
        // In this barrier design the parent simply also participates in
        // Here workers produce and push their routing tables to shared c
        send_own_table_to_neighbors(self); // pushes messages to shared c

        // Now each router must receive messages from ALL adjacent router
        // Either: pull until adj_count messages received (from shared_qu
        // or wait until an adjacency-specific queue has that many messag

```

```

    receive_from_neighbors(self); // must ensure thread-safe queue ac
    // After receive, wait for all nodes to finish their receive-phase
    barrier_wait(&exchange_barrier);

    // Phase 2: update using received vectors
    update_routing_table(self);
    // Mark entries updated this iteration (store a flag per entry if

    // Wait for all nodes to finish update-phase:
    barrier_wait(&update_barrier);

    // (Optional) Parent / coordinator can print tables after update_
    it++;
}
}

/* parent/coordinator thread (optional) */
void coordinator() {
    // If you want the parent to control iteration timing (2 seconds betw
    // make parent also participate in the same barriers, or coordinate p
    for (int it=0; it<MAX_ITR; ++it) {
        // Wait 2 seconds before starting exchange (if required by spec)
        sleep(2);
        // Let workers run exchange; they use barrier_wait to synchronize
        // After exchange_barrier completes (i.e., all nodes have receive
        // parent can optionally print intermediate state.
        // After update_barrier completes, print routing tables with aste
        // sleep(2) between rounds if spec requires.
    }
}
}

```

Notes on the pseudocode above:

- `barrier_wait` is reusable and safe if all threads call it exactly once per phase. Resetting `count = 0` inside the critical section prevents stale counts.
- `threshold` for each barrier should be the number of participants. If parent coordinates but does not participate, `threshold = number of node threads`.
- `receive_from_neighbors(self)` must only return after `self->adj_count` messages meant for `self` are read; use per-link messages tagged with destination.

Extra suggestions & correctness tips

- **Use per-edge messages or tag messages with (src,dst,iteration)** so a node ignores old/duplicate messages.
 - **Include iteration number in queue messages** so late messages from previous rounds are ignored.
 - **Avoid global `rcv_cnt` used by multiple nodes.** Use per-phase barrier counts (one counter in barrier).
 - **Mark updated entries:** inside `update_routing_table()` compare old vs new value and set a `changed` flag for printing. Reset that flag before next iteration.
 - **Consider using an existing barrier API** (e.g., `pthread_barrier_t` in POSIX) to avoid implementing your own unless required for learning.
 - **If you keep parent-driven booleans (`xchg_init` , `updatation_init`)** then parent must set them while holding the corresponding lock, broadcast `cond_broadcast` and workers must wait with `while (!xchg_init) cond_wait(...)` . But using a barrier is cleaner.
-